



My 2025 Dropbox CodeSignal OA: Real Questions & Secrets to Success

 **Charlie** · November 8, 2025 · 8 min read



My Dropbox OA Experience

I first heard about Dropbox's CodeSignal online assessment (OA) when recruiting reached out after I applied. Compared to the typical "random LeetCode churn," this felt like a paced, progressive challenge: several questions in one sitting, difficulty ramping up, and the freedom to switch between tasks. CodeSignal publicly documents that its General Coding Assessment (GCA) uses 4 questions in ~60 minutes with flexible time allocation and "best-score kept" across submissions.

Dropbox explicitly uses CodeSignal as the first step for many early-career SWE roles to fairly evaluate core coding skills before interviews.



typically proceed to technical/behavioral rounds, sometimes including a code-review/debugging session. My timeline followed that playbook almost exactly.

I am really grateful for the tool Linkjob.ai, and that's also why I'm sharing my entire interview experience here. Having an invisible AI assistant during the interview is indeed very convenient.

 Timely AI Help for Interviews and Online Assessment

Best AI Interview Copilot: 100% Undetectable, Ultra-Low Latency and Coding Help

Linkjob.ai guarantees 100% that it won't be detected by screen-sharing in live interview and coding assessment.

Dive In Now !

Watch a 1 min Demo

Assessment Structure

Dropbox uses this test to see how well you can code in real situations. The coding challenges connect to each other and finish one part then go to the next part.

Duration	Number of Questions
60 minutes	Up to 4 questions



MOVE ON.

Key Features

The dropbox codesignal test has many kinds of problems. The questions might like these:

- Single-function
- Progressive single-function
- Database
- DevOps
- Quiz
- Filesystem
- Free-coding
- Frontend
- Writing
- Flutter SDK

It allow use any popular programming language. The platform lets you pick from many languages, so I do not need to learn a new one for this test.

Dropbox CodeSignal Assessment Process



Dropbox CodeSignal OA Questions

The Dropbox CodeSignal assessment contained three coding problems ranging from easy to medium difficulty. The topics spanned backtracking, sliding-window statistics and graph traversal. Below I explain the requirements, solution strategies and complexity analyses for each, along with snippets of my own code.

Question 1: Word Pattern II



such that if each character in `pattern` is replaced by the string it maps to, then the resulting string is `s`. A **bijective mapping** means that no two characters map to the same string, and no character maps to two different strings.

Example 1:

Input: `pattern = "abab", s = "redblueredblue"`

Output: `true`

Explanation: One possible mapping is as follows:

`'a' -> "red"`

`'b' -> "blue"`

Example 2:

Input: `pattern = "aaaa", s = "asdadasdasd"`

Output: `true`

Explanation: One possible mapping is as follows:

`'a' -> "asd"`

Example 3:

Input: `pattern = "aabb", s = "xyzabcxzyabc"`

Output: `false`

In this problem you're given a pattern string and another string `s`, and you must decide if there is a bijective mapping between each character in the pattern and a **non-empty substring** of `s`. The mapping is bijective: different pattern characters must map to different substrings, and vice versa. For example, the pattern `"abab"` can match `"redblueredblue"` if `'a' -> "red"` and `'b' -> "blue"`.

Solution strategy

The natural approach is a depth-first search with backtracking:

1. **Track mappings and used substrings.** Use a dictionary to map pattern characters to substrings and a set to keep track of which substrings are already taken. This enforces the one-to-one correspondence.
2. **Recursive matching.** For the current character in the pattern, if it is already in the map, check whether the next characters in `s` match the assigned substring; if so, recurse on the remainder of the pattern and string. If the character isn't mapped yet, iterate over all possible non-empty prefixes of the remaining string and try mapping the



3. **Backtracking.** If a mapping fails, undo it and try the next candidate substring. If both the pattern and the string are consumed at the same time, return `True` ; if one finishes before the other, return `False` .

Complexity analysis

In the worst case, each character can map to many different substrings, so the search tree grows exponentially. Let n be the length of s and m the length of the pattern; then the time complexity is roughly $O(n^m)$. The space complexity comes from the recursion stack, the dictionary and the set, which is $O(m + n)$.

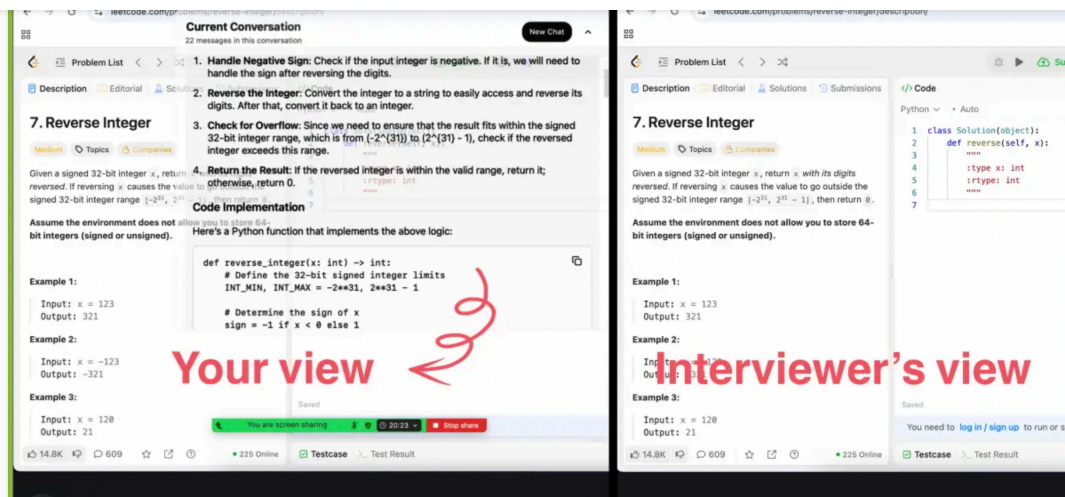
My implementation

Below is my Python backtracking solution. It defines a `dfs` function that tries to assign a substring to each new pattern character and then recurses. Failed assignments are removed to backtrack. I used the Linkjob.ai tool, and this answer was generated in just a few seconds.



```
def dfs(self, pattern, str, dict):  
    if len(pattern) == 0 and len(str) > 0:  
        return False  
    if len(pattern) == len(str) == 0:  
        return True  
    for end in range(1, len(str)-len(pattern)+2): # +2 because it is the "end of an end"  
        if pattern[0] not in dict and str[:end] not in dict.values():  
            dict[pattern[0]] = str[:end]  
            if self.dfs(pattern[1:], str[end:], dict):  
                return True  
            del dict[pattern[0]]  
        elif pattern[0] in dict and dict[pattern[0]] == str[:end]:  
            if self.dfs(pattern[1:], str[end:], dict):  
                return True  
    return False
```

When I encountered this issue, it caught me off guard. I utilised the linkjob.ai tool, which remains invisible on screen and thus evades detection by interview platforms.



Question 2: Design Hit Counter

2.Design Hit Counter

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).

Your system should accept a `timestamp` parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., `timestamp` is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the `HitCounter` class:

- `HitCounter()` Initializes the object of the hit counter system.
- `void hit(int timestamp)` Records a hit that happened at `timestamp` (in seconds). Several hits may happen at the same `timestamp`.
- `int getHits(int timestamp)` Returns the number of hits in the past 5 minutes from `timestamp` (i.e., the past 300 seconds).

Example 1:

Input

```
["HitCounter", "hit", "hit", "hit", "getHits", "hit", "getHits", "getHits"]
[[], [1], [2], [3], [4], [300], [300], [301]]
```

Output

```
[null, null, null, null, 3, null, 4, 3]
```

Explanation

```
HitCounter hitCounter = new HitCounter();
hitCounter.hit(1);    // hit at timestamp 1.
hitCounter.hit(2);    // hit at timestamp 2.
hitCounter.hit(3);    // hit at timestamp 3.
hitCounter.getHits(4); // get hits at timestamp 4, return 3.
hitCounter.hit(300);  // hit at timestamp 300.
hitCounter.getHits(300); // get hits at timestamp 300, return 4.
hitCounter.getHits(301); // get hits at timestamp 301, return 3.
```

```
public class HitCounter {
```




Methods: `hit(timestamp)` records a hit at the given second, and `getHits(timestamp)` returns how many hits occurred in the last 300 seconds. Timestamps are provided in non-decreasing order.

Solution strategy

1. **Queue as a sliding window.** Because you only care about hits in the last 300 seconds, keep a queue of timestamps. When you call `hit(t)`, append `t` to the end. When you call `getHits(t)`, remove timestamps from the front that are older than `t - 299`, then return the queue length.
2. **Why it's efficient.** Timestamps in the queue remain sorted; outdated entries are removed only during queries. Each timestamp is enqueued and dequeued at most once, so the amortized time per operation is $O(1)$.
3. **Space usage.** In the worst case, if there's at least one hit every second, the queue holds at most 300 timestamps, so the space usage is $O(300)$ – effectively constant.

My implementation

Here's my Java implementation of a `HitCounter` class. It uses a `LinkedList` to store timestamps. The `hit` method enqueues the timestamp, and `getHits` discards outdated entries and returns the current size.



```
Queue<Integer> q = null;

/** Initialize your data structure here. */
public HitCounter() {
    q = new LinkedList<Integer>();
}

/** Record a hit.

@param timestamp - The current timestamp (in seconds granularity). */
public void hit(int timestamp) {
    q.offer(timestamp);
}

/** Return the number of hits in the past 5 minutes.

@param timestamp - The current timestamp (in seconds granularity). */
public int getHits(int timestamp) {
    while(!q.isEmpty() && timestamp - q.peek() >= 300) {
        q.poll();
    }
    return q.size();
}
}
```

Question 3: Max Area of Island



The **area** of an island is the number of cells with a value `1` in the island.

Return the **maximum area** of an island in `grid`. If there is no island, return `0`.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: `grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,0,0],[0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,0,0,0],[0,0,0,0,0,0,0,1,1,0,0,0]]`

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: `grid = [[0,0,0,0,0,0,0]]`

Output: 0

In this problem you're given an $m \times n$ binary matrix `grid` of zeros (water) and ones (land). An island is a group of ones connected **four-directionally** (up, down, left, right). You need to find the area (number of cells) of the largest island. If there are no islands, return 0.

Solution strategy

- 1. Traverse every cell.** Iterate through each coordinate (r, c) ; whenever `grid[r][c] == 1`, start a depth-first search (DFS) from that cell to compute the island's area.
- 2. Depth-first search.** Visit the current cell, mark it as visited (by setting it to 0 or adding to a `visited` set), then recursively explore its four neighbors (up, down, left, right), adding one for each land cell encountered.
- 3. Track the maximum.** After each DFS call, update a global `max_area` with the maximum of its current value and the returned island size.



The recursion stack can grow as large as the number of land cells in an island (up to $m \times n$), giving a space complexity of $O(m \times n)$.

My implementation

The following Python code uses DFS to compute the area of each island. It tracks visited cells in a set to avoid revisiting nodes; alternatively, you can mutate the grid in place to mark visited cells.

```
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        visited = set()
        max_island = 0

        def dfs(r, c):
            if r not in range(rows) or c not in range(cols) or (r, c) in visited or grid[r][c] == 0:
                return 0
            visited.add((r, c))
            return 1 + dfs(r+1, c) + dfs(r-1, c) + dfs(r, c+1) + dfs(r, c-1)

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1 and (r, c) not in visited:
                    max_island = max(max_island, dfs(r, c))

        return max_island
```

Practical Tips for the Dropbox CodeSignal OA

After working through the four-level OA and the final interview, here are some lessons I'd share with other candidates:

Step-by-Step Debugging

In an online platform like CodeSignal, debugging time is precious. Break your program into small functions and print intermediate variables to validate each step. For DFS problems, for example, print the current



Simplified Modeling Strategy

Many interview problems have elaborate backstories but simple cores. For the Hit Counter, you might think you need to tally every event, but a simple queue gives you everything you need. Word Pattern II feels open-ended until you realize you can prune the search space aggressively by storing mappings and used substrings. When you read a problem, strip away the narrative to uncover the underlying algorithm.

Safety-First Optimization

Under time pressure, always get a correct solution first. If you're unsure about bounds in a binary search, choose a conservative upper limit. If your initial approach is slow but straightforward, implement it to pass the basic tests, then optimize if time remains. A working answer collects points and gives you a base to improve on.

Key Preparation Tips for Dropbox CodeSignal OA

Targeted Practice by Question Type

- **Backtracking and string mapping.** Problems like Word Pattern II rely on recursive search with pruning. Practise similar pattern-matching and permutation problems to become comfortable with building and backtracking from a mapping.
- **Queues and sliding windows.** The Hit Counter is essentially a fixed-window frequency query. Strengthen this skill by solving problems such as "sliding window maximum," "moving average" or "longest substring without repeating characters."
- **Graph traversal.** Max Area of Island tests your ability to explore grids using DFS or BFS. Do related problems like "Number of



Time Management Tips

You only get 60 minutes for the whole technical assessment. You need to use your time well. Read each question quickly, but do not rush. If you get stuck, move to the next part and come back later if you have time. Keep an eye on the clock. Try to finish each question in about 15 minutes.

Here is a simple plan:

Task	Time Limit
Read instructions	2 minutes
Solve each problem	15 minutes
Review answers	8 minutes

Note: Save a few minutes at the end to check your code for mistakes.

Reading & Code Review Practice

Because Dropbox's final interview includes a code-review round, spend time reading other people's code. Pick a few open-source projects or LeetCode solutions, review them critically and write short comments on what they do well and what you would improve. In the interview, it's more important to communicate your observations and trade-offs clearly than to fix every minor issue.



What happens after the OA?

In my round, passing the CodeSignal assessment led to a short HR call and then a final interview with three parts: a coding problem (medium-difficulty LeetCode style), a code review (analyse and comment on someone else's code) and a behavioural interview.

Which programming languages are allowed?

CodeSignal supports many popular languages, including Python, Java, C++, JavaScript and more. Use the language you're most comfortable with to reduce the chance of syntax mistakes.

Which practice problems should I focus on?

For backtracking, try problems like "Permutation II" and "Word Pattern II." For sliding windows, try "Moving Average" and "Longest Substring Without Repeating Characters." For graph traversal, practise "Number of Islands" and "Flood Fill." Also, complete practice questions on CodeSignal to familiarize yourself with their interface.

See Also

[Top Alternatives to Coding Interview Copilot for Coders](#)

[My Experience With Final Round AI: Disappointing Results](#)

[Top Alternatives to AI Apply for Interview Assistance](#)

[Leading Alternatives to Parakeet AI for Interview Help](#)

[Codesignal Questions](#)



Real-Time AI Interview Copilot AI Coding Interview Copilot

Interview Questions

© Copyright 2025 Linkjob.ai - All Rights Reserved.